DAW_M03_T14_ACT14_CristianCantero

En esta actividad voy a crear un programa que solicite al usuario utilizar un menu para insertar, actualizar, borrar, listar todos los alumnmos o salir del programa.

Voy a empezar documentando la actividad desde el momento que me descargo los drivers de la BDD hasta como configuro la misma.
Como ya llevamos muchos trabajos construyendo clases con sus atributos, getters/setters, etc.. la parte donde creo la clase Alumno la saltaré y trabajaré dando por sentado que está ya está creada durante todo el ejercicio.

Empezamos.

## PARTIENDO DE 0

Mi estación de trabajo es un Macbook ProM2. El ID que utilizo para programar es Java Netbeans 25. La BDD que he escogido es MySQL. Previamente no tengo nada instalado.

## LA INSTALACIÓN

Después de comprobar que versión tenía instalada la version 5.0.8 de Homebrew (y más que nada por saber si tenia el homebrew instalado tambíen) procedí a instalar MySQL con el comando:

- brew Install mysql

```
cristiancantero@MacBook-Pro-de-Cristian ~ % brew install mysql
==> Auto-updating Homebrew...
Adjust how often this is run with `$HOMEBREW_AUTO_UPDATE_SECS` or disable with
`$HOMEBREW_NO_AUTO_UPDATE=1`. Hide these hints with `$HOMEBREW_NO_ENV_HINTS=1` (se
e `man brew`).
==> Downloading https://ghcr.io/v2/homebrew/core/portable-ruby/blobs/sha256:f41c72
b891c40623f9d5cd2135f58a1b8a5c014ae0414988828940931627 6c72
################################################################## 100.0%
```

```
All done!
cristiancantero@MacBook-Pro-de-Cristian ~ % mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 9.6.0 Homebrew

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

Siguiendo los pasos y creando de 0 una nueva instalación le puse una contraseña y me devolvió un All done!

Para comprobar si estaba todo correcto ejecuté en el terminal:

- *mysql -u root -p*

Introduje el password que había configurado y a funcionar.

```
mysql>
```

Ahora puedo controlar por el terminal acciones como USE o DROP propias del lenguaje sql para usar la BDD.

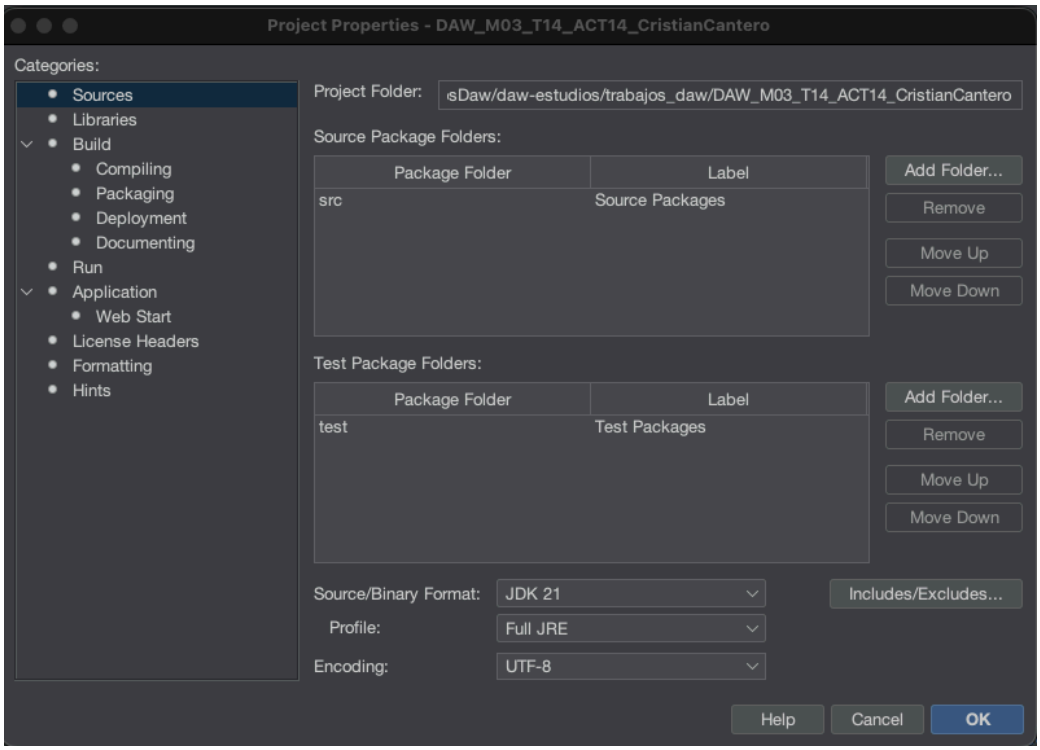
```
mysql> USE escuela;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
mysql> DROP TABLE alumnos;
Query OK, 0 rows affected (0,016 sec)
```

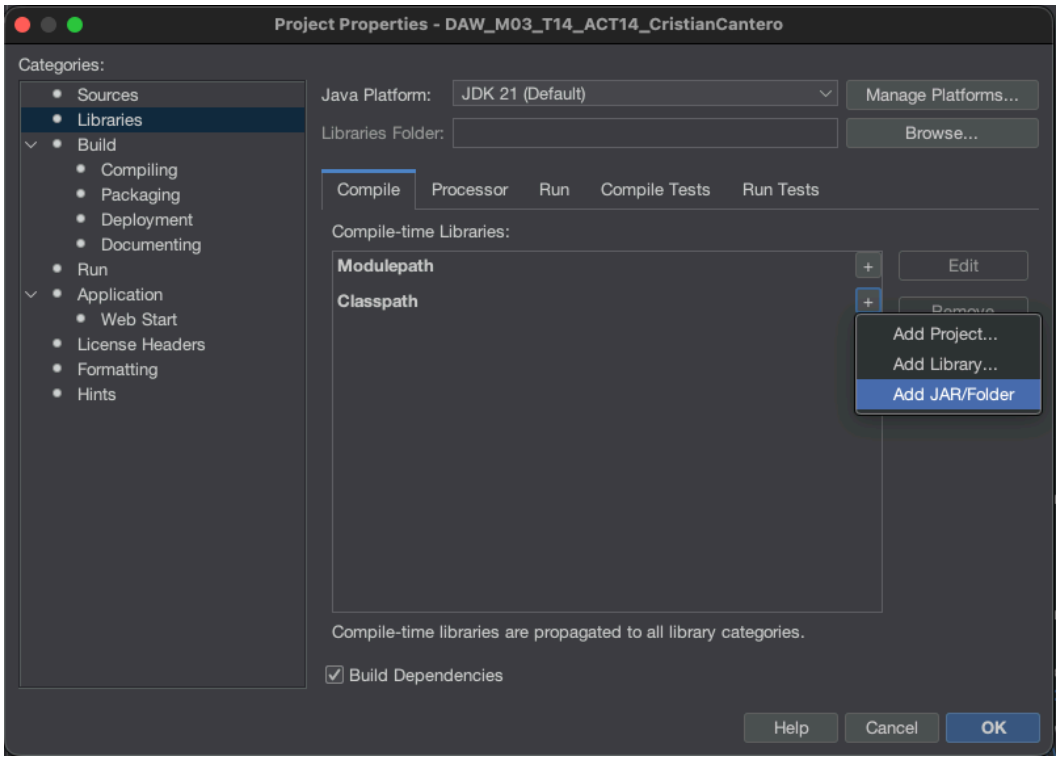
Una vez la instalación completada voy a conectar el proyecto de java netbeans a la BDD. Esta acción es imprescindible si quiero que mi programa pueda trabajar con la misma.

## LA CONEXIÓN DEL PROYECTO A LA BDD

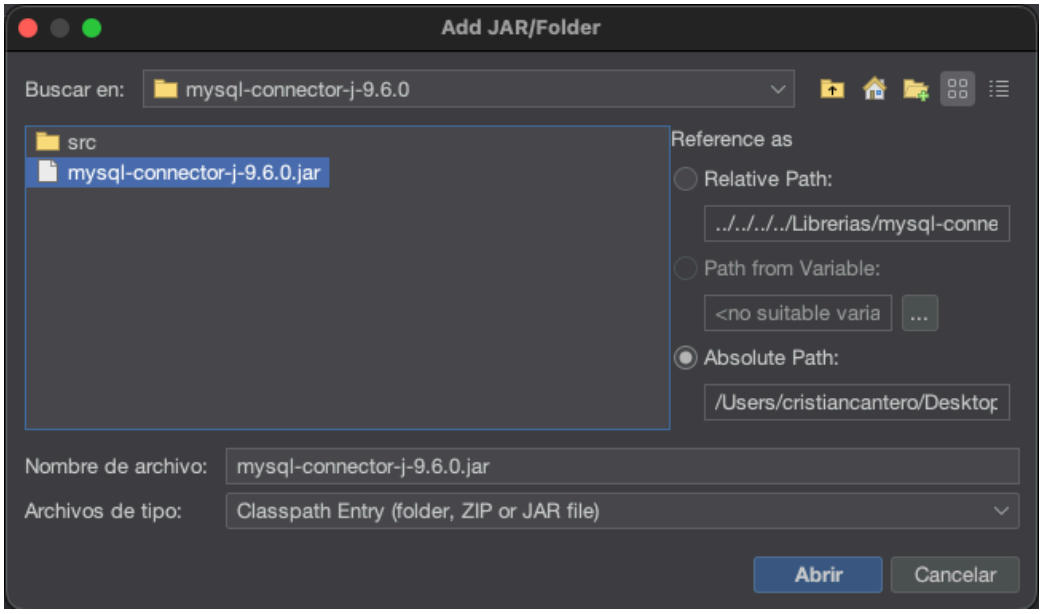
Hago click derecho sobre el proyecto y deslilzo el puntero hasta Properties haciendo click. Se desplega esta ventana. 👇🏾



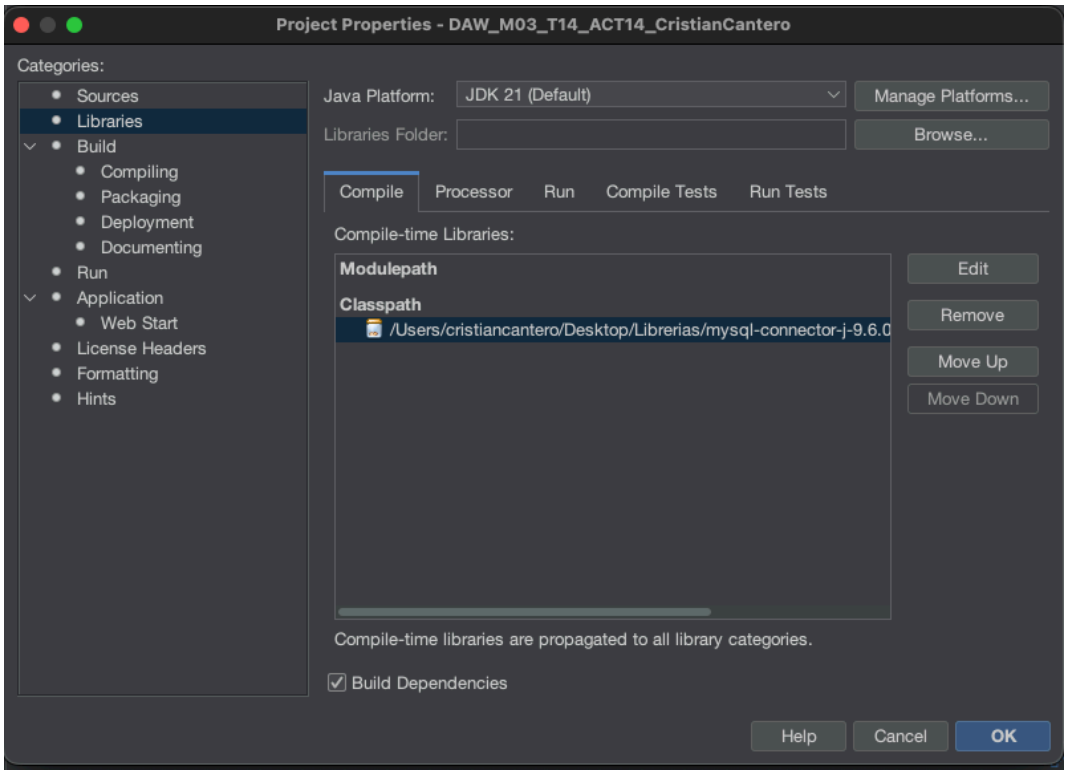

En la lista de la derecha busco Libraries. Y en las pestañas del centro, en la pestaña Compile hay dos compile-time Libraries. Modulepath y Classpath. El segundo es el que nos interesa. Le clico al simbolo '+' y luego Add JARFolder.



Ya tenía descargado el mysql-connector así que le clico al .jar




Mini comprobación antes del ok…




Y listos, ya deberíamos tener nuestra bdd linkada con el proyecto.
Para saber si la base de datos está correctamente conectada voy hacer una comprobación que me ha enseñado my friend chatGPT.

## COMPROBACIÓN DE LA CONEXIÓN

En GestsionBDD pongo este código que establece una conexión con el servidor de base de datos MySQL utilizando la API JDBC.

```java
public class GestionBDD {
    public static void main(String[] args) {

        try{
        String url = "jdbc:mysql://localhost:3306/";
        String user = "root";
        String password = "12345678";

        Connection con = DriverManager.getConnection(url, user, password);
        }catch(Exception e){
            e.printStackTrace();
        }
    }


}
```

Primero defino 3 variables: la URL de conexión (url), el user (user) y la contraseña(password). La URL incluye el localhost, que hace referencia al propi equipo, y el puerto 3306, que es el puerto por defecto de MySQL.

Después uso el método DriverManager.getConnection() para abrir la conexion con la base de datos. Este método devuelve un objeto de tipo connection, que permitirá ejecutar sentencias SQL posteriormente.

Todo el proceso se encuentra dentro de un bloque try-catch, ya que la conexion puede fallar (por ejemplo, si la contraseña es incorrecta o el servidor no está activo). En caso de error, se captura la excepción y se muestra po rconsola mediente e.printStackTrace(), lo que facilita la deteccion de problemas.

Ahora puedo crear el método que va a generar la BDD.

```
public static void crearBDD(Connection con){
    try{
        String sql = "CREATE DATABASE IF NOT EXISTS escuela;";
        con.createStatement().executeUpdate(sql);
    }catch(Exception e){
        e.printStackTrace();
    }
}
```

En una variable que llamo sql guardo la sentencia para que cree la base de datos. La siguiente línea es la que ejecuta la orden.

*con.createStatement().executeUpdate(sql);*

*con* -> concexión con la base de datos. En la línea 21 tenemos la creación d esta variable.
*createStatement()* -> crea un objeto para enviar ódrdenes sql
*executeUpdate(sql)* -> ejecuta la orden sql

La función termina con el catch.

Ahora puedo continuar con el try anterior creando primero la BDD con el método crearBDD(con);

```
    Connection con = DriverManager.getConnection(url, user, password);
        crearBDD(con);
        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/escuela",user,password);
        System.out.println("conexión establecida...");
    }catch(Exception e){
        e.printStackTrace();
    }
}
```

Añado la BDD y después una de las líneas más importantes del ejercicio.

*con = DriverManager.getConnection("jdbc:mysql:///localhost:3306/escuela",user,password);*

*DriverManager* —> clase que gestiona conexiones
*getConnection(…)* —> método que abre la conexión
*“jdbc:mysql://localhost:3306/escuela”* —> dirección de la base de datos
*user* —> usuario
*password* —> contraseña

Acuerdate que al principio del try  url, user y password están definidas. Después las hemos usado para establecer conexion en esta línea de código que acabo de documentar anteriormente.

Para probar si la base está establecida hago un System.out.println con un mensaje de comprobación.

```
run:
conexión establecida...
BUILD SUCCESSFUL (total time: 0 seconds)
```

El mensaje salta y aunque no es una prueba feaciente de que la conexión está establecida se que no me ha saltado ninguna excepción. Por lo tanto la conexión tiene que funcionar.

La mejor prueba para comprobar la conexión es ejecutar esta sentencia:

*con.createStatement().executeQuery(“SELECT 1”);*
*System.out.println(“Mensaje confirmación”);*

Con la conexión establecida continuo con el ejercicio.

## CREAR METODO PARA LA TABLA

```
public static void crearTablaAlumnos(Connection con){
    try{
        String sql = "CREATE TABLE IF NOT EXISTS alumnos ("+
                "id INT AUTO_INCREMENT PRIMARY KEY, " +
                "nombre VARCHAR(50), "+
                "edad INT, "+
                "curso VARCHAR(50), " +
                "media DOBUBLE)";
        con.createStatement().executeUpdate(sql);
        System.out.println("tabla creada");
    }catch(Exception e){
        e.printStackTrace();
    }
}
```

Este método contiene un String con toda la tabla sql. Una vez está la tabla insertada usamos la conexion para enviar la orden sql:

*con.createStatement().executeUpdate(sql);*

*con* —> conexión activa con MySQL
*createStatement()* -> crea un objeto para enviar órdenes
*executeUpdate(sql)* -> ejecuta la orden sql. De esta orden puede ejecutar CREATE, INSERT, UPDATE y DELETE. Pero no para SELECT.

## METODOS PEDIR E INGRESAR ALUMNO

Para que el programa pueda ingresar alumnos utilizaré dos métodos. Uno que solicita al usuario los alumnos y otro que los inserta en la base de datos.

*ingresarAlumno()*

```java
public static Alumno ingresarAlumno(){
    Scanner sc = new Scanner (System.in);

    System.out.println("Nombre: ");
    String nombre = sc.nextLine();
    System.out.println("Edad: ");
    int edad = sc.nextInt();
    sc.nextLine();
    System.out.println("curso: ");
    String curso = sc.nextLine();
    System.out.println("media: ");
    Double media = sc.nextDouble();

    return new Alumno(0,nombre,edad,curso,media);


}
```

Este método crea y devuelve un objeto de tipo Alumno.

El atributo id se inicializa con el valor 0 al crear el objeto ya que la basa de datos lo genera automáticamente mediante AUTO_INCREMENT. Por eso luego no lo usaré para la inserción.

*insertarAlumno(Connection con)*

```java
public static void insertarAlumno(Connection con){
    try{
        Alumno a = ingresarAlumno();
        String sql = "INSERT INTO alumnos (nombre, edad, curso, media) VALUES (" +
                "'" + a.getNombre() + "', " +
                a.getEdad() + ", '" +
                a.getCurso() + "', " +
                a.getMedia() + ")";
        con.createStatement().executeUpdate(sql);
        System.out.println("Alumno insertado con éxito");
    }catch(Exception e){
        e.printStackTrace();
    }
}
```

Este método recibe un objecto Connection que representa la conexión activa con la base de datos y permite ejecutar sentencias sql.
El código de inserción está envuelto en un try/catch por si la ejecución de las sentencias sql dan error. Si hubiera fallo se captura y se muestra por consola para facilitar su detección.

Alumno a = pedirAlumno();

Hace que ingresarAlumno() se ejecute y Alumno a guarda el resultado.
Ejecutamos la sentencia sql y usamos el método get para insertar los datos. La manera de formatearlo tiene su forma de ser pensada:
Como esa sentencia la guardamos en una variable String que la llamamos sql organizamos un orden para el formateo del texto.

Lo que va entre "…" esta comillas. Es Java, el texto fijo.
Lo que va entre '…' estas comillas. Es sql.
+ son las variables de Java

Ejemplo:

INSERT INTO alumnos(nombre, edad, curso, media) VALUES('Cristian', 49, '2.B', 9.5);

Ahora hay que envolverlo entre las comillas de Java y sustituir los valores de VALUES por los valores que obtenemos con get.

```
String sql =
"INSERT INTO alumnos(nombre, edad, curso,media)
VALUES ("+" ' "+a.getNombre()+" ' ," +a.getEdad()+", ' "+getCurso()+"'," +getMedia()+")";
```

Una vez hecho el insert se ejecuta la conexión.

*con.createStatement().executeUpdate(sql);*

Termino controlando la excepción con el catch.

## ACTUALIZAR ALUMNO

```java
public static void actualizarAlumno(Connection con, Scanner sc) {
    try {
        System.out.println("Introduce el ID del alumno a modificar: ");
        int idModificar = Integer.parseInt(sc.nextLine());
        System.out.println("Introduce de nuevo el nombre: ");
        String nombreNuevo = sc.nextLine();
        System.out.println("Introduce la edad: ");
        int edad = Integer.parseInt(sc.nextLine());
        System.out.println("Introduce el curso: ");
        String curso = sc.nextLine();
        System.out.println("Introduce la media");
        double media = Double.parseDouble(sc.nextLine());

        String sql = "UPDATE alumnos SET "
                + "nombre = '" + nombreNuevo + "', "
                + "edad = " + edad + ", "
                + "curso = '" + curso + "', "
                + "media = " + media
                + " WHERE id = " + idModificar;

        int filas = con.createStatement().executeUpdate(sql);

        if (filas > 0) {
            System.out.println("Alumno actualizado correctamente.");
        } else {
            System.out.println("No existe un alumno con ese ID");
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Este método recibe como parámetro la conexión a la BDD y un objeto scanner para que el usuario introduzca los datos.
Dentro del try, se pregunta al usuario el id del usuario a modificar. Seguidamente y sin más se piden los nuevos datos y todos quedan guardados en sus variables.
Guardo la sentencia SQL en un String. La sentencia contiene UPDATE alumnos SET que sirve para actualizar y dejar actualizados los datos dentro de la base de datos. El String queda formateado con la sentencia SQL y las variables con los datos introducidos por el usuario.
Declaro un int filas con la conexión a la base de datos.
Lógica de programación: si filas es mayor que 0 el alumno se actualilzó correctamente. Si no, no existe un alumno con ese ID.
Acabo con el catch para controlar la excepción.

**BORRAR ALUMNO**

```java
public static void borrarAlumno(Connection con, Scanner sc) {
    try {
        System.out.println("Introduce el ID del alumno a borrar");
        int idBorrar = Integer.parseInt(sc.nextLine());

        String sql = "DELETE FROM alumnos WHERE id = " + idBorrar;
        int filas = con.createStatement().executeUpdate(sql);

        if (filas > 0) {
            System.out.println("Alumno borrado correctamente");
        } else {
            System.out.println("No existe un alumno con este ID");
        }

    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Este método lleva por parametros la conexión a la BDD y un objeto scanner para introducir datos por el usuario.
Dentro del try:
Se pregunta al usuario la ID del alumno que quiere borrar y se guarda en una variable.
Declaro un String sql con la sentencia Delete (borrar) de dentro de alumnos donde el id sea igual al dato introducido por el usuario.
Utilizo el mismo método de antes para que el método se ejecute en la BDD. Es decir, ejecuto la conexión en un String que llamo sql y con un condicional compruebo si el alumno se ha borrado correctamente o si bien no existe un alumno con ese id. Terminamos con el catch.

Vamos con el último método.

## LISTAR ALUMNOS

```java
public static void listarAlumnos(Connection con) {
    try {
        String sql = "SELECT * FROM alumnos";
        ResultSet rs = con.createStatement().executeQuery(sql);

        while (rs.next()) {
            System.out.println(
                    rs.getInt("id") + " - "
                    + rs.getString("nombre") + " - "
                    + rs.getInt("edad") + " - "
                    + rs.getString("curso") + " - "
                    + rs.getDouble("media")
            );
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

Este método recibe la conexión por parámetros. Dentro del try ejecuta una String con la sentencia sql SELECT * FROM alumnos; que significa selecciona todo de alumnos. Después ejecuto el código sql y lo guardo en una varible de tipo ResultSet, que es un objeto que almacena el resultado de una consulta SQL, permitiendo recorrer y acceder a los datos obtenidos. Para trabajar con ResultSet es necesario su import => import java.sql.ResultSet;

Voy hacer un desglose:

con —> conexión a la base de datos
createStatement() —> preparo la orden SQL
executeQuery(sql) —> ejecuto un SELECT
ResultSet rs —> guardo los resultados de la bdd

¿Como se usa?

```
While (rs.next()){
        System.out.println(rs.getInt("Id"));
}
```

—> rs.next() —> pasa a la siguiente fila
—> rs.get…()—> lee los datos

El while dice que mientras se cumpla la condición que deje pasar a la siguiente fila (rs.next) imprime los datos (rs.get…)
Se termina con el catch.

Detalle importante:
executeQuery() solo se usa para SELECT
executeUpdate() solo se usa para INSERT, UPDATE, DELETE

**BLOQUE DEL OUTPUT**

Llego la hora de la verdad. Vamos a ver como funciona el programa.
—>Run file

```
run:
tabla creada
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
1
Nombre:
Cristian
Edad:
69
curso:
1
media:
9.5
java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
        at java.base/java.util.Scanner.next(Scanner.java:1602)
        at java.base/java.util.Scanner.nextDouble(Scanner.java:2573)
        at ACT14.GestionBDD.ingresarAlumno(GestionBDD.java:112)
        at ACT14.GestionBDD.insertarAlumno(GestionBDD.java:120)
        at ACT14.GestionBDD.main(GestionBDD.java:46)
```

La primera en la frente. Puse un punto en vez de coma en media y saltó un InputMisMatchException. Error apuntado. Vamos a volver hacer lo mismo pero usando una coma en vez de un punto para representar un double.

```
run:
tabla creada
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
1
Nombre:
Cristian
Edad:
49
curso:
1
media:
9,5
Alumno insertado con éxito
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
1
Nombre:
Juan
Edad:
45
curso:
2
media:
8,9
Alumno insertado con éxito
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
```

Por el momento me dejó ingresar dos alumnos. Como observación es curioso, pero ahora cuando el programa salta no se detiene. Tenemos el error en pantalla pero me devuelve a la página principal.

```
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
4
2 - Cristian - 49 - 1 - 9.5
3 - Juan - 45 - 2 - 8.9
4 - Pepa - 34 - 2 - 9.9
```

Ingreso un alumno más y compruebo usando la opción 4. Es curioso porque empieza a contar por la id 2… supongo que habrán quedado restos de la id 1 cuando introduje mal los datos del primer alumno.

Vamos a probar actualizar y borrar.

```
2 - Cristian - 49 - 1 - 9.5
3 - Juan - 45 - 2 - 8.9
4 - Pepa - 34 - 2 - 9.9
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
2
Introduce el ID del alumno a modificar:
2
Introduce de nuevo el nombre:
Pepe
Introduce la edad:
34
Introduce el curso:
2
Introduce la media
9.1
Alumno actualizado correctamente.
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
4
2 - Pepe - 34 - 2 - 9.1
3 - Juan - 45 - 2 - 8.9
4 - Pepa - 34 - 2 - 9.9
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
```

===> Aquí se ve claramente que con ID 2 está Cristian

Se ingresa la opción 2 y se siguen las instrucciones

Alumno actualizado y comprobamos

Ahora Cristian es Pepe y menos la ID todo a cambiado

```
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
3
Introduce el ID del alumno a borrar
2
Alumno borrado correctamente
===Menú principal===
1. Ingresar alumno
2. Actualizar alumno
3. Borrar alumno
4. Listar alumnos
0. Salir
4
3 - Juan - 45 - 2 - 8.9
4 - Pepa - 34 - 2 - 9.9
```

Ingresamos la opción 3 borrar alumno y como podemos ver:

Hay uno que ha desaparecido

## CONCLUSIONES

Un trabajo chulisimo sobre todo por el tema de conectar la base de datos y trabajar con la misma base de datos usando lenguaje SQL desde Java. El formateo del texto ha sido un poco de dolor de cabeza hasta que lo he entendido. Y el proceso de trabajo usando createStatement().executeQuery() solo paraselect o executeUpdate(nombre_variable) para hacer Instert, update o delete ha sido bastante chulo. Claro ahora me queda saber que papel juegan aqui los arraylist, los hashmap, etc.. Como siempre la conclusión es la misma: **hay que estudia mucho.**

## BIBLIOGRAFIA

Apuntes Linkia (el de la base de datos es infumable), chat GPT.